Atividade prática ATVII

Professor Dr. Eng. Gerson Penha

Introdução:

Prezados empreendedores e empreendedoras, após a imersão no domínio da indústria criativa e a construção do primeiro microsserviço da plataforma cinebridge, é chegada a hora de refletirmos sobre uma questão fundamental que permeia toda a engenharia de software: a qualidade estrutural do sistema que concebemos. A atividade que se inicia agora nos convida a compreender, em profundidade, os riscos inerentes a um sistema mal projetado, as origens e os propósitos dos padrões de projeto, e os benefícios concretos que eles trazem para a longevidade e o sucesso de qualquer empreendimento tecnológico. Esta reflexão não é meramente acadêmica, mas sim uma necessidade prática para qualquer empreendedor que deseje construir soluções robustas, escaláveis e economicamente viáveis.

A construção de um sistema de software é, em muitos aspectos, análoga à construção de uma edificação. Uma fundação mal calculada, a utilização de materiais inadequados ou a ausência de um projeto estrutural coerente podem levar, a curto ou médio prazo, ao colapso da obra. No universo digital, as consequências de um projeto descuidado são igualmente severas, embora muitas vezes menos visíveis a olho nu. Um sistema mal projetado apresenta sintomas clássicos e devastadores. Em primeiro lugar, a rigidez: qualquer alteração, por menor que seja, exige a modificação de múltiplos pontos do código, tornando o sistema frágil e difícil de evoluir. Em segundo lugar, a fragilidade: uma correção em uma parte do sistema introduz erros imprevistos em partes completamente distintas, num efeito dominó que consome tempo e recursos preciosos da equipe de desenvolvimento. Em terceiro lugar, a imobilidade: o sistema torna-se tão emaranhado que é impossível reutilizar seus componentes em outros contextos, forçando a equipe a reinventar a roda a cada novo projeto.

Estes problemas, que parecem meramente técnicos, traduzem-se em prejuízos financeiros e estratégicos significativos. A dívida técnica, conceito que descreve o custo implícito de escolhas de projeto subótimas, cresce exponencialmente em sistemas desestruturados. Cada nova funcionalidade demanda mais tempo e esforço para ser implementada, a curva de aprendizado para novos desenvolvedores torna-se íngreme, e a taxa de erros em produção se eleva, corroendo a confiança dos usuários e aumentando os custos de suporte e manutenção. Em um cenário empreendedor, onde a agilidade e a capacidade de resposta ao mercado são diferenciais competitivos, um sistema mal projetado é um passivo que drena o caixa da empresa e compromete sua capacidade de inovar.

Foi justamente para enfrentar este conjunto de desafios recorrentes que emergiu, na engenharia de software, o conceito de padrões de projeto. A história dos padrões de projeto não se inicia, curiosamente, na computação, mas na arquitetura e no urbanismo da década de 1970, com os trabalhos do arquiteto Christopher Alexander. Alexander observou que certos problemas de design em construções e cidades surgiam repetidamente e que, para cada um deles, existiam soluções testadas e comprovadas que poderiam ser documentadas e reutilizadas. Ele propôs, então, uma linguagem de padrões, um catálogo de boas práticas que capturavam a sabedoria acumulada por gerações de construtores. Esta ideia revolucionária foi transplantada para o desenvolvimento de software na década de 1990, quando a programação orientada a objetos ganhava força e os desenvolvedores se deparavam com problemas estruturais semelhantes em projetos distintos. A necessidade de reduzir o acoplamento,

organizar responsabilidades, criar sistemas extensíveis e lidar com mudanças constantes eram desafios universais, e as soluções para esses desafios começaram a ser catalogadas.

O marco definitivo dessa catalogação foi a publicação, em 1994, do livro "Design Patterns: Elements of Reusable Object-Oriented Software", pelos autores Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, o quarteto conhecido como Gang of Four. Esta obra organizou vinte e três padrões de projeto, classificando-os em três categorias principais: criacionais, que tratam dos mecanismos de criação de objetos; estruturais, que se ocupam da composição de classes e objetos para formar estruturas maiores; e comportamentais, que focam na comunicação e atribuição de responsabilidades entre objetos. Mais do que uma simples lista de soluções, o livro estabeleceu uma linguagem comum para que engenheiros de software pudessem comunicar decisões de projeto de forma eficiente, discutir trade-offs e aplicar soluções consagradas com confiança. A contribuição dos padrões de projeto transcendeu a mera técnica, promovendo uma mudança na forma de pensar o desenvolvimento de software, incentivando os profissionais a identificar problemas recorrentes e a avaliar conscientemente as consequências de cada escolha arquitetural.

Os benefícios da aplicação diligente de padrões de projeto são numerosos e bem documentados pela experiência da indústria. Em primeiro lugar, eles promovem a organização e a limpeza do código, tornando-o mais legível e compreensível para toda a equipe. Esta clareza é um pré-requisito para a manutenção eficiente, uma vez que facilita a localização de defeitos e a implementação de novas funcionalidades sem o receio de quebrar o funcionamento existente. Em segundo lugar, os padrões de projeto incentivam a coesão e o baixo acoplamento, características que permitem escalonar o projeto com maior facilidade, adicionando novas capacidades sem reescrever partes inteiras do sistema. Em terceiro lugar, eles atuam como um antídoto contra a proliferação de códigos repetitivos, evitando a duplicação de lógica que não apenas infla o sistema, mas também multiplica os pontos onde erros podem ser introduzidos. Padrões como a Injeção de Dependência, inspirada nos padrões criacionais, e o padrão Observador, que aplicamos na primeira atividade, tornaram-se tão fundamentais que muitos frameworks e APIs modernas, como Spring e .NET, já os incorporam em sua própria arquitetura, abstraindo a complexidade de sua implementação direta .

A ausência desta disciplina de projeto já custou caro a diversas organizações ao redor do mundo. Embora seja raro encontrar relatos públicos detalhados de falhas catastróficas exclusivamente atribuídas à falta de padrões, é consenso na indústria que sistemas legados mal escritos, com rotinas excessivamente amarradas, representam um dos maiores desafios para a evolução tecnológica de empresas consolidadas. Nestes casos, a simples manutenção de uma funcionalidade existente exige a cópia e colagem de códigos parecidos em várias classes diferentes, um processo moroso e propenso a erros. Grandes instituições financeiras, seguradoras e operadoras de telecomunicações, que possuem sistemas centrais desenvolvidos há décadas, frequentemente dedicam orçamentos anuais bilionários exclusivamente à modernização de suas plataformas, um custo direto da dívida técnica acumulada por escolhas de projeto inadequadas no passado. Casos emblemáticos de instabilidade em sistemas de reservas de companhias aéreas, falhas em plataformas de negociação de ações e interrupções em serviços de entrega, embora frequentemente atribuídos a sobrecarga ou erros operacionais, têm suas raízes em uma arquitetura de software frágil, onde uma falha em um componente se propaga de forma incontrolável devido ao alto acoplamento. Estes exemplos, embora não nominalmente citados em manuais, são estudados nas melhores escolas de engenharia de

software como advertências sobre o que acontece quando a qualidade estrutural é negligenciada.

Neste sentido, a segunda atividade do nosso percurso empreendedor consistirá em uma análise crítica do microsserviço de recomendação que construímos na primeira etapa. Os empreendedores deverão identificar, no código existente, pontos de rigidez, fragilidade ou imobilidade. Em seguida, deverão propor e implementar refatorações baseadas em padrões de projeto adicionais, que não os já aplicados, com o objetivo de melhorar a qualidade estrutural e a preparação para mudanças. Esta atividade visa consolidar a compreensão de que os padrões de projeto não são receitas mágicas, mas ferramentas de concepção que, utilizadas com critério, transformam um sistema que funciona em um sistema que perdura, gerando valor real para o negócio e reduzindo o risco tecnológico do empreendimento.

Atividade:

Prezados empreendedores e empreendedoras, após a conclusão da primeira atividade, na qual implementamos o microsserviço de recomendação e orquestração de equipes com os padrões Strategy, Template Method, Observer e Visitor, é chegado o momento de dar um passo adiante na evolução da plataforma cinebridge. A segunda atividade consistirá em um ciclo de refatoração e extensão do sistema existente, com o objetivo de introduzir três novos padrões de projeto: Prototype, Factory Method e Abstract Factory. Esta etapa não é meramente um exercício acadêmico, mas sim uma necessidade prática imposta pela expansão dos requisitos de negócio e pela necessidade de tornar o sistema mais flexível, configurável e preparado para lidar com a diversidade de projetos e profissionais que a plataforma atrairá.

O contexto que motiva esta refatoração é o seguinte. Até o momento, nosso microsserviço trabalhava com um modelo único de projeto e de profissional, com atributos fixos e um conjunto estável de papéis. Contudo, a experiência prática no setor audiovisual independente revela que diferentes gêneros de produção, como documentários, animações, videoclipes e longas-metragens, exigem estruturas de projeto radicalmente distintas, com diferentes conjuntos de papéis, diferentes métricas de avaliação e diferentes regras de orçamento. Além disso, a plataforma cinebridge pretende atender não apenas a produtores individuais, mas também a estúdios de médio porte e a agências de publicidade, cada qual com seus próprios fluxos de trabalho e critérios de sucesso. A criação de novas instâncias de projeto, com todas as suas especificidades, tornou-se uma operação repetitiva e propensa a erros no código atual, uma vez que cada novo tipo de projeto exige a duplicação de lógica de criação e validação.

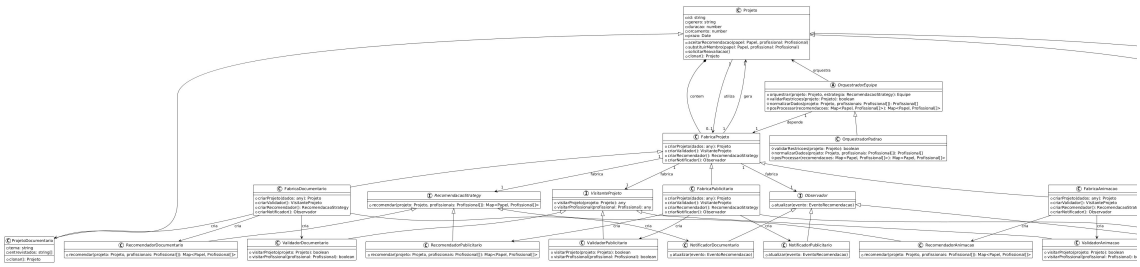
Para resolver este problema, implementaremos o padrão Prototype, que nos permitirá clonar objetos complexos sem acoplá-los às suas classes concretas. Cada tipo de projeto, como ProjetoDocumentario, ProjetoAnimacao ou ProjetoPublicitario, poderá ser registrado como um protótipo inicial, a partir do qual novas instâncias serão criadas por clonagem, com possibilidade de ajustes finos em seus atributos específicos. Este padrão reduzirá a complexidade da criação de novos projetos, especialmente quando estes compartilham grande parte de sua estrutura interna, e permitirá que a plataforma ofereça modelos pré-configurados para diferentes cenários, acelerando o processo de cadastro de novos projetos pelos produtores.

Paralelamente, enfrentamos o desafio de instanciar, de maneira consistente, as famílias de objetos relacionados que compõem um projeto completo. Cada projeto não é uma entidade isolada, mas um agregado que inclui seu próprio conjunto de papéis, suas próprias regras de validação, seu próprio motor de recomendação ajustado e seu próprio formato de notificação. O padrão Factory Method será empregado para delegar a criação de cada um desses objetos a

métodos especializados, de modo que subclasses de Projeto possam definir como seus componentes são fabricados. Por exemplo, um projeto de animação criará uma fábrica concreta que retorna um validador de consistência específico para animações, um recomendador que prioriza habilidades em software de modelagem 3D e um gerador de relatório com métricas de eficiência de renderização.

Por fim, adotaremos o padrão Abstract Factory para garantir que todas as famílias de objetos relacionados a um mesmo tipo de projeto sejam criadas em harmonia, sem inconsistências. A fábrica abstrata definirá uma interface para criar o projeto completo, o validador, o recomendador e o notificador, enquanto cada fábrica concreta, como FabricaDocumentario ou FabricaAnimacao, produzirá objetos que sabem cooperar entre si. Isto assegurará que, ao trocarmos o tipo de projeto, todo o ecossistema de objetos se adapte automaticamente, sem necessidade de condicionais espalhadas pelo código. O resultado será um sistema muito mais coeso, onde a adição de um novo tipo de projeto, como produções para realidade virtual, exigirá apenas a criação de uma nova fábrica concreta, sem modificar o núcleo do orquestrador.

Para orientar os empreendedores nesta empreitada, apresento o diagrama de classes revisado, em preto e branco, com todas as cardinalidades e tipos explicitamente definidos. Este diagrama incorpora os novos padrões e serve como guia arquitetural para as alterações a serem implementadas.



A proposta de trabalho para esta segunda atividade é a seguinte. Os empreendedores deverão, a partir do código existente do microsserviço de recomendação, realizar as seguintes tarefas. Em primeiro lugar, refatorar a hierarquia de classes de Projeto para implementar o padrão Prototype, garantindo que cada subclasse forneça um método clonar que retorne uma cópia profunda de si mesma, incluindo todos os objetos agregados, como a lista de papéis e as restrições específicas. Em segundo lugar, definir a interface FabricaProjeto e implementar as três fábricas concretas para os tipos documentário, animação e publicitário, cada qual responsável por criar o projeto correspondente, bem como seu validador, seu recomendador e seu notificador específicos. Em terceiro lugar, modificar o OrquestradorEquipe para receber, em seu construtor, uma instância de FabricaProjeto, e utilizar essa fábrica para obter todos os objetos necessários durante a orquestração, eliminando quaisquer condicionais baseadas no tipo de projeto que porventura existam. Em quarto lugar, ajustar o SistemaRecomendacao para que, ao receber uma solicitação de recomendação, identifique o tipo de projeto e selecione a fábrica adequada, utilizando-a para criar todo o ecossistema de objetos necessário para a execução. Por fim, implementar testes de integração que demonstrem que, ao alternar entre diferentes fábricas, o sistema produz recomendações, validações e notificações consistentes com o tipo de projeto, sem que nenhuma linha de código do orquestrador precise ser alterada para acomodar um novo tipo.

Esta atividade exigirá dos empreendedores uma compreensão profunda das relações entre os padrões criacionais e comportamentais, bem como a habilidade de reorganizar uma base de código existente sem quebrar funcionalidades já entregues. O desafio é significativo, mas o resultado será um sistema muito mais preparado para crescer, acolhendo novos tipos de projetos com um esforço mínimo e sem risco de regressão. Acredito que esta experiência consolidará a visão de que a arquitetura de software não é um fim em si mesma, mas um meio para garantir que o empreendimento possa evoluir com agilidade e segurança, entregando valor contínuo aos seus usuários. Aceitem este desafio com o espírito de engenheiros e empreendedores que entendem que a qualidade estrutural é o alicerce de qualquer negócio digital de sucesso.

Por fim, para que o sistema possa ser comercializado com facilidade ele de ser compatível com as plataformas Windows 10 ou superior, Linux Ubuntu 24.04.03 ou superior e distribuições Linux derivadas do Ubuntu.